

CS-470 Homework 3

High-Level Synthesis

Ricardo Perello Mas

2026-05-20

Conventions used in this writeup

- An operation with latency L issued at cycle t produces its output **at cycle** $t + L$, where it can be consumed by a downstream operation.
- Latencies (from the assignment):
 - Integer: load/store = 1, add = 1, multiply = 4
 - Float: load/store = 1, add = 3, multiply = 5
- Each array maps to one BRAM with **one memory port** unless explicitly partitioned.
- Pragma syntax follows Vitis HLS 2025.2 (all optional parameters specified explicitly).
- N denotes `ARRAY_SIZE`.
- In Gantt charts: rows are loop iterations, columns are clock cycles. LD/ST are 1-cycle memory ops; MUL and ADD span as many columns as their latency.

1 Static HLS (Section 2.1)

1.1 Kernel 1 — `array[i] = array[i] * 5`

Modified code

```
1 void kernel1(int array[ARRAY_SIZE]) {
2     #pragma HLS array_partition variable=array type=cyclic factor=3 dim=1
3     for (int i = 0; i < ARRAY_SIZE; i++) {
4         #pragma HLS pipeline II=1 rewind
5         int x = array[i];
6         array[i] = (x << 2) + x;
7     }
8 }
```

Per-iteration dataflow

A single iteration i performs three operations:

1. LD `array[i]` — 1 cycle
2. ADD `(x << 2) + x` — 1 cycle (constant shift is wiring)
3. ST `array[i]` — 1 cycle

Per-iteration latency = $1 + 1 + 1 = 3$ cycles.

Scheduling Gantt chart — first 3 iterations ($II = 1$)

cycle:	0	1	2	3	4
iter 0:	LD	ADD	ST		
iter 1:		LD	ADD	ST	
iter 2:			LD	ADD	ST

With $II = 1$ a new iteration starts every cycle; the constant multiply is implemented as a shift plus one add.

Initiation Interval

$II = 1$.

The body needs **2 memory operations per iteration** (1 load + 1 store) but a single-port BRAM only services **1 access per cycle**, so $\text{ResMII}_{\text{naïve}} = 2$. To get $II = 1$ we apply cyclic partitioning with factor 3:

- Iteration i uses bank $i \bmod 3$ for both its LD (at cycle i) and its ST (at cycle $i + 2$).
- At cycle c the active accesses are: LD of iteration c on bank $c \bmod 3$, and ST of iteration $c - 2$ on bank $(c - 2) \bmod 3$.
- These banks are always different, so the port constraint is satisfied.
- A cyclic factor of 2 would conflict here, because the shorter shift-add pipeline would make LD of iteration c and ST of iteration $c - 2$ hit the same parity bank.

No loop-carried dependence (each iteration touches a distinct address), so $\text{RecMII} = 0$ and the binding II is $\text{ResMII} = 1$ after partitioning.

Cycle counts

- One iteration: 3 cycles.
- Whole task (N iterations, $II = 1$): $(N - 1) + 3 = N + 2$ cycles.

Resources used

Resource	Count	Notes
Multipliers	0	Constant multiply is rewritten
Adders	1	Computes $(x \ll 2) + x$
BRAM ports for array	3	One port per bank after partition

Optimizations and trade-offs

Optimization	Why	Downside
pipeline II=1	Overlaps consecutive iterations. Without it each iteration serialises (3N total).	Pipeline registers between stages.
(x << 2) + x	Removes the 4-cycle integer multiplier and shortens the pipeline fill/drain from 6 cycles to 3 cycles.	Uses one adder; relies on constant shifts being wiring.
array_partition cyclic factor=3	Gives enough memory banks for the LD of iteration i and the ST of iteration $i - 2$ to proceed in the same cycle.	Uses three BRAM banks instead of one. A lower-area alternative is keeping the multiplier and using factor 2, which still has $II = 1$ but total latency $N + 5$.

1.2 Kernel 2 — `array[i] = array[i-1] + array[i-2] * array[i-3]`

Modified code

```

1 void kernel2(int array[ARRAY_SIZE]) {
2     // Preload the three-element sliding window from memory
3     // (cost ignored per assignment: outside the loop).
4     int v0 = array[0]; // plays the role of array[i-3]
5     int v1 = array[1]; // plays the role of array[i-2]
6     int v2 = array[2]; // plays the role of array[i-1]
7
8     for (int i = 3; i < ARRAY_SIZE; i++) {
9         #pragma HLS pipeline II=1 rewind
10        int result = v2 + v1 * v0;
11        array[i] = result;
12        // Shift the window: every register update is a free wire,
13        // no memory traffic involved.
14        v0 = v1;
15        v1 = v2;
16        v2 = result;
17    }
18 }

```

Per-iteration dataflow (after the rewrite)

The naïve code performs 3 loads, 1 multiply, 1 add, and 1 store per iteration. With a single BRAM port the three loads alone push ResMII to 3, and there is a hard loop-carried dependence: iteration i writes `array[i]`, which iteration $i + 1$ reads as `array[(i+1)-1]`. The synthesiser conservatively waits for that store to commit, blowing II up to ≈ 9 cycles.

Key observation: **every value the loop reads, it has already read or computed in a recent iteration.** So we keep a 3-deep sliding window in registers (`v0`, `v1`, `v2`) and only ever store to memory. After the rewrite:

1. MUL `v1 · v0` — 4 cycles
2. ADD `v2 + MUL` — 1 cycle (carries `v2 = previous result`)
3. ST `array[i]` — 1 cycle
4. Shift the window: `v0 ← v1; v1 ← v2; v2 ← result` (free).

Per-iteration latency = $4 + 1 + 1 = 6$ cycles.

Scheduling Gantt chart — first 3 iterations ($II = 1$)

cycle:	0	1	2	3	4	5	6	7	8
iter 0:	MUL	----	----	----	ADD	ST			
iter 1:		MUL	----	----	----	ADD	ST		
iter 2:			MUL	----	----	----	ADD	ST	

The carried dependence is $ADD_i \rightarrow ADD_{i+1}$: **result_i** feeds the next iteration's **v2**. Latency 1 cycle $\Rightarrow$ RecMII = 1.

Initiation Interval

$II = 1$.

- ResMII: one ST per iteration on a single port $\Rightarrow 1$. No more loads inside the loop.
- RecMII: **result_i** $\rightarrow$ **v2** $\rightarrow$ ADD_{i+1} . Path length = 1-cycle ADD $\Rightarrow 1$.
- $II = \max(1, 1) = 1$.

Note: the MUL of iteration $i + 1$ uses the **previous** window's **v1**, **v2** (which become the new **v0**, **v1**), so the multiplier chain has **no carried dependence** — it is fully pipelined despite its 4-cycle latency.

Cycle counts

- Loop runs from $i = 3$ to $i = N - 1 \Rightarrow N - 3$ iterations.
- One iteration: 6 cycles.
- Whole task: $(N - 3 - 1) + 6 = N + 2$ cycles.

Resources used

Resource	Count	Notes
Multipliers	1	Pipelined, busy every cycle in steady state
Adders	1	1-cycle integer add on the recurrence
BRAM ports for array	1	Only stores remain inside the loop
Registers	3 (32-bit)	Sliding window v0 , v1 , v2

Optimizations and trade-offs

Optimization	Why	Downside
Sliding-window registers for array[i-1] , array[i-2] , array[i-3]	Eliminates three loads per iteration, frees the BRAM port, and breaks the synthesised RAW chain between iterations' ST/LD.	Requires preloading the first three elements (free here) and a small register file.
pipeline $II=1$	Issues a new MUL each cycle so the 4-cycle multiplier is fully utilised.	Standard pipeline register cost.
No unroll	Unroll by 2 would double multiplier and adder count with no II benefit (RecMII already 1) — pure waste.	—

A subtle pitfall: in the unmodified code the synthesiser sees `array[i-1]` and could in principle prove that the index differs from `i` and forward through registers automatically. In practice Vitis HLS does not infer the sliding window from such code — it must be written explicitly.

1.3 Kernel 3 — Histogram: `hist[index[i]] = hist[index[i]] + weight[i]`

Modified code

```

1 void kernel3(float hist[ARRAY_SIZE],
2             float weight[ARRAY_SIZE],
3             int index[ARRAY_SIZE]) {
4     for (int i = 0; i < ARRAY_SIZE; ++i) {
5         #pragma HLS pipeline II=5 rewind
6         hist[index[i]] = hist[index[i]] + weight[i];
7     }
8 }

```

Per-iteration dataflow

Operations per iteration:

1. LD `index[i]` — 1 cycle (index port)
2. LD `weight[i]` — 1 cycle (weight port, in parallel)
3. LD `hist[index[i]]` — 1 cycle (hist port, must wait for the index)
4. FADD `hist + weight` — 3 cycles (float add)
5. ST `hist[index[i]]` — 1 cycle (hist port)

cycle	op
0	LD <code>index[i]</code> , LD <code>weight[i]</code> (parallel, different arrays)
1	LD <code>hist[index[i]]</code> (now we have the address)
2–4	FADD
5	ST <code>hist[index[i]]</code>

Per-iteration latency = 6 cycles.

Scheduling Gantt chart — first 3 iterations (`II = 5`)

cycle:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	15														
iter 0:	LDi	LDh	FADD	----	----	ST									
	LDw														
iter 1:						LDi	LDh	FADD	----	----	ST				
						LDw									
iter 2:											LDi	LDh	FADD	----	
	----	ST													
											LDw				

LDi = LD `index[i]`, LDw = LD `weight[i]`, LDh = LD `hist[index[i]]`.

Initiation Interval

$II = 5$.

The key issue is that the memory dependence through `hist` is not generally false. If `index[i] == index[i+1]`, iteration $i + 1$ must read the value written by iteration i ; otherwise the histogram update loses one of the weights. Since the assignment does not state that the indices are unique, we must preserve this possible RAW dependence.

With the safe schedule:

- ResMII: `hist` is hit twice per iteration (1 LD + 1 ST) on a single port $\Rightarrow 2$.
- `weight` and `index` each see one LD/iter — no bottleneck.
- RecMII: the worst-case same-bin recurrence is LD `hist` $\rightarrow$ FADD $\rightarrow$ ST `hist` $\rightarrow$ next LD `hist`. With the schedule above, the store of iteration i occurs at cycle $i + 5$, and the next iteration's `hist` load can occur at cycle $i + 6$. Thus the next loop start is 5 cycles after the previous one.
- $II = \max(2, 5) = 5$.

Could we use a `dependence false` pragma to push lower? Only under an external precondition, such as all overlapping iterations targeting distinct bins. Under that precondition the carried dependence disappears and the single `hist` port would bind the loop at $II = 2$. For a general histogram, however, that pragma changes the program's meaning when indices repeat, so I do not use it in the correctness-preserving solution.

Cycle counts

- One iteration: 6 cycles.
- Whole task (N iterations, $II = 5$): $(N - 1) \cdot 5 + 6 = 5N + 1$ cycles.

Resources used

Resource	Count	Notes
Multipliers	0	—
Float adders	1	3-cycle FADD, pipelined
BRAM ports for <code>hist</code>	1 (2 ops/iter)	True dependence possible when indices repeat
BRAM ports for <code>weight</code>	1 (1 op/iter)	—
BRAM ports for <code>index</code>	1 (1 op/iter)	—

Optimizations and trade-offs

Optimization	Why	Downside
<code>pipeline II=5</code>	Allows the independent <code>index</code> and <code>weight</code> loads of the next iteration to overlap with the previous iteration's final <code>hist</code> store, while preserving same-bin correctness.	The true histogram recurrence still dominates; speedup over the fully sequential loop is small.
Do not use <code>false by default</code>	Repeated indices are legal for a histogram, so the inter-iteration RAW through <code>hist</code> may be real.	Gives up the conditional $II = 2$ schedule that would be valid only for a no-collision input contract.
Cannot safely partition <code>hist</code>	Indices come from <code>index[i]</code> at runtime — no static cyclic/block partition can rule out two overlapping iterations hitting the same bank/bin.	—

1.4 Kernel 4 — accumulator update with two indexed reads

Modified code

```

1 void kernel4(int array[ARRAY_SIZE], int index[ARRAY_SIZE], int offset) {
2     // Hoist the loop-carried accumulator out of memory.

```

```

3   int acc = array[offset];
4   // Bootstrap the sliding-window register for array[i].
5   int a_curr = array[offset + 1];
6
7   for (int i = offset + 1; i < ARRAY_SIZE - 1; ++i) {
8       #pragma HLS pipeline II=1 rewind
9       int a_next = array[i + 1];           // one LD per iteration
10      int idx     = index[i];               // one LD on the other port
11      int diff    = a_next - a_curr;        // SUB
12      int delta   = idx * diff;             // MUL -- only one per
                                           // iteration
13      acc         = acc + delta;            // recurrence on acc
14      a_curr      = a_next;                // shift the register
15  }
16
17  array[offset] = acc;                     // single ST outside the loop
18 }

```

Per-iteration dataflow (after the rewrite)

The original body has four loads (`array[offset]`, `index[i]`, `array[i]`, `array[i+1]`), two multiplies, two adds/subtracts, and one store back to `array[offset]` — and every iteration round-trips the accumulator through memory. Three rewrites collapse this:

1. **Hoist `array[offset]` to a scalar `acc`.** The same address is read and written every iteration $\Rightarrow$ the synthesiser would chain `LD` $\rightarrow$ `FADD` $\rightarrow$ `ST` $\rightarrow$ `LD` across iterations ($\text{RecMII} \approx 7$). Lifting the accumulator into a register breaks that chain — only the `ADD acc + delta` cycle remains.
2. **Sliding register for `array[i]`.** Iteration $i+1$ reads `array[(i+1)] = array[i+1]`, which iteration i already loaded. Carry it forward in `a_curr`.
3. **Factor the two multiplies:** $-idx \cdot a_i + idx \cdot a_{i+1} = idx \cdot (a_{i+1} - a_i)$. One multiply, one extra subtract. We trade a 4-cycle `MUL` for a 1-cycle `SUB`.

After all three:

1. `LD array[i+1]`, `LD index[i]` (parallel) — 1 cycle
2. `SUB a_next - a_curr` — 1 cycle
3. `MUL idx · diff` — 4 cycles
4. `ADD acc + delta` — 1 cycle (the carried op)

Per-iteration latency = $1 + 1 + 4 + 1 = 7$ cycles.

Scheduling Gantt chart — first 3 iterations ($II = 1$)

cycle:	0	1	2	3	4	5	6	7	8	9
iter 0:	LDx LDi	SUB	MUL	----	----	----	ADD			
iter 1:		LDx LDi	SUB	MUL	----	----	----	ADD		
iter 2:			LDx LDi	SUB	MUL	----	----	----	ADD	

`LDx` = `LD array[i+1]`, `LDi` = `LD index[i]`. The carried dependence is $\text{ADD}_i \rightarrow \text{acc} \rightarrow \text{ADD}_{i+1}$, length = 1 cycle $\Rightarrow \text{RecMII} = 1$.

Initiation Interval

$II = 1$.

- ResMII: `array` sees 1 LD/iter, `index` sees 1 LD/iter. Separate ports $\Rightarrow$ 1.
- RecMII: ADD on the accumulator path, latency 1 $\Rightarrow$ 1.
- $II = \max(1, 1) = 1$.

The 4-cycle MUL is **not** on the carried path — it consumes iteration i 's `idx` and `diff` and feeds only that iteration's ADD. With a pipelined multiplier we issue one MUL per cycle and it has no bearing on II.

Cycle counts

Iterations executed: $M = (\text{ARRAY_SIZE} - 1) - (\text{offset} + 1) = N - \text{offset} - 2$.

- One iteration: 7 cycles.
- Whole task (M iterations, $II = 1$): $(M - 1) + 7 = M + 6 = N - \text{offset} + 4$ cycles (the final ST is outside the loop and negligible per spec).

Resources used

Resource	Count	Notes
Multipliers	1	Factored 2 muls into 1: <code>idx · (a_next - a_curr)</code>
Adders	2	1 SUB in the body, 1 ADD on the recurrence
BRAM ports for <code>array</code>	1	1 LD/iter after the rolling register
BRAM ports for <code>index</code>	1	—
Registers	2	<code>acc</code> , <code>a_curr</code> (plus pipeline registers)

Optimizations and trade-offs

Optimization	Why	Downside
Scalar accumulator (<code>acc</code>) replaces LD/ST of <code>array[offset]</code>	Removes the dominant carried chain (LD $\rightarrow$... $\rightarrow$ ST $\rightarrow$ LD) and frees the <code>array</code> port. Without this, $\text{RecMII} \geq 7$.	None functionally — semantically equivalent because no other code writes <code>array[offset]</code> during the loop.
Rolling register <code>a_curr</code> for <code>array[i]</code>	Halves the load traffic on <code>array</code> . Without it, two LDs/iter on a single port would force $\text{ResMII} = 2$.	Tiny: one extra 32-bit register and bootstrap load.
Factor the two multiplies into <code>idx · (a_next - a_curr)</code>	Saves a multiplier (1 mul instead of 2) and shortens the compute (4-cycle MUL $\rightarrow$ 1-cycle SUB).	None — algebraically identical.
pipeline $II=1$	Standard.	Standard.

1.5 Kernel 5 — Data-dependent loop exit

```

1 float kernel5(float bound, float a[ARRAY_SIZE], float b[ARRAY_SIZE]) {
2     int i = 0;
3     float sum = 0;
4     while (sum < bound && i < ARRAY_SIZE) {
5         sum = a[i] + b[i];
6         i++;
7     }
8     return sum;
9 }
```


Modified code

```

1 float kernel5(float bound, float a[ARRAY_SIZE], float b[ARRAY_SIZE]) {
2     int i = 0;
3     float sum = 0;
4     while (sum < bound && i < ARRAY_SIZE) {
5         #pragma HLS pipeline II=5 rewind
6         sum = a[i] + b[i];
7         i++;
8     }
9     return sum;
10 }

```

Per-iteration dataflow

1. LD a[i] and LD b[i] in parallel — 1 cycle (different arrays, different ports)
2. FADD a + b — 3 cycles
3. FCMP sum < bound — 1 cycle (no separate latency given in spec; treated as 1-cycle compare)
4. && with i < N and branch — free

Per-iteration latency = 1 + 3 + 1 = 5 cycles.

If the course model treats the final comparison/branch as purely combinational because comparisons are not listed in the latency table, replace this with 4 cycles. I count it as 1 cycle here to be conservative and to make the control-dependence cost explicit.

Scheduling Gantt chart — first 3 iterations (static, II = 5)

cycle:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
iter 0:	LDa	FADD	----	----	CMP										
	LDb														
iter 1:					LDa	FADD	----	----	CMP						
					LDb										
iter 2:									LDa	FADD	----	----	CMP		
									LDb						

Each iteration must finish its compare before the next iteration is allowed to begin: the synthesiser cannot start iteration $i + 1$'s loads while iteration i 's exit condition is in flight, because it does not know whether iteration $i + 1$ should execute at all.

Initiation Interval

$II = 5$.

- ResMII: 1 LD per port per iteration on disjoint arrays $\Rightarrow 1$.
- RecMII: **the loop exit condition is the binding recurrence**. Path = LD $\rightarrow$ FADD $\rightarrow$ FCMP = 1 + 3 + 1 = 5 cycles $\Rightarrow 5$.
- $II = \max(1, 5) = 5$.

The pipeline pragma is included for completeness — Vitis HLS will still report $II = 5$ because the recurrence dominates. Without the pragma the design would simply run sequentially (5 cycles/iter as well). There is essentially no static-HLS optimisation beyond confirming this lower bound.

Note that `sum` is **overwritten** every iteration (`sum = a[i] + b[i]`, not `sum += ...`). So the only loop-carried value is `i` (trivial) and the exit decision itself. The whole bottleneck is control, not data.

Cycle counts

Let K be the iteration count at which the loop exits ($K \leq N$).

- One iteration: 5 cycles.
- Whole task (K iterations, $\Pi = 5$): $\approx 5K$ cycles.

Under the zero-latency-compare convention, the corresponding static count is approximately $4K$.

For comparison, a dynamically scheduled implementation could approach $K + 4$ cycles (see Section 2).

Resources used

Resource	Count	Notes
Multipliers	0	—
Float adders	1	3-cycle FADD, pipelined
Float compare	1	< against bound
BRAM ports for a	1 (1 LD/iter)	—
BRAM ports for b	1 (1 LD/iter)	—

Why no further static-HLS optimization is possible

Every other knob (`unroll`, `array_partition`, `dependence false`) either does not apply or does not help:

- `unroll` of a `while` whose iteration count depends on runtime data is unsafe — the synthesiser would need to assume some upper bound and add fix-up code, with no Π benefit because the exit decision still serialises iteration groups.
- `array_partition` — already maximal: `a` and `b` are on separate BRAMs, and there is only 1 access per iteration on each.
- `dependence false` — there is no memory dependence to relax; the bottleneck is **control flow**, not memory.

The fundamental obstruction is that a static schedule must, at compile time, decide whether to execute iteration $i + 1$ — and it cannot, because the answer depends on runtime data. The only way to overlap iterations is to **speculate** that the loop continues and roll back on misspeculation, which by definition is dynamic HLS.

2 Dynamic HLS (Section 2.2) — Kernel 5

The shape of the dynamic circuit

A dynamically scheduled HLS implementation builds a dataflow circuit where each operator fires the moment its inputs are valid, communicating via the SELF (Synchronous Elastic Flow) handshake (`valid/ready`). The loop body becomes a small dataflow graph with a **Merge** for `i` at the loop entry, a **Branch** at the exit, and **Buff/FIFO** insertion to keep the recurrence cycle deadlock-free and high-throughput.

For Kernel 5 the branch at the bottom is replaced by a **Speculator** (**Spec.** **Branch**): it predicts “the loop continues” and emits a new iteration’s tokens *before* the actual `sum < bound` comparison resolves. **Save** units sit on the boundary where regular tokens enter the speculative region (the `i++` feedback edge and the `a[i]/b[i]` token-issue paths) so that on a misprediction the originals can be re-emitted. **Commit** units sit on the output edges so any misspeculated tokens that leak through downstream are squashed before reaching the exit.

Why this beats the static design

Static HLS has $II = 5$ because each iteration's start is gated on the previous iteration's compare. A speculative dataflow circuit issues new iterations every cycle — the only floor is now the resource-MII (one load per port per iteration $\Rightarrow \text{ResMII} = 1$). The cycle diagrams in the course slides (the same `do { d = a[i]+b[i]; ... } while (d < x)` example) show exactly this contrast:

- **Non-speculative dynamic schedule:** still ≈ 5 cycles between iteration starts because the **Branch** waits on the compare.
- **Speculative dynamic schedule ($II = 1$):** iterations begin every cycle. When iteration K 's compare finally fires “exit”, iterations $K + 1, K + 2, K + 3, K + 4$ are already in-flight; the speculator squashes them via the **Commit** units and the loop terminates.

Approximate cycle counts:

- Static HLS: $\approx 5K$ cycles.
- Speculative dynamic HLS: $\approx K + 4$ cycles in the no-misprediction case. The “+4” covers the in-flight iterations that get squashed.

With a zero-latency compare convention, these become roughly $4K$ and $K + 3$.

For large K this is close to a $5\times$ **speedup** in steady state.

Where it really shines: a favorable data distribution

Consider an input where `sum < bound` is true for almost every iteration until the very end. Concretely, suppose:

- `bound = 1.0e6`
- `a[i], b[i] ∈ [1.0, 10.0]` for all $i < N - 1$
- The last entries are crafted so that `a[N-1] + b[N-1] > bound`

Then:

- The loop-continuation condition `sum < bound` is **true** for almost every iteration, so the speculator's “continue” prediction is correct $K - 1$ times in a row and wrong only once (the final iteration).
- The misprediction cost is bounded: at most 4 in-flight speculative iterations are squashed, regardless of K .
- Total cycles $\approx K + 4$, versus $5K$ in static HLS — a near- $5\times$ speedup that **grows with K** .

Note that `sum = a[i] + b[i]` (assignment, not accumulation) means each iteration's compare is **independent of the previous iterations' compares**. The speculator therefore does not have to predict a chain of branches — it only has to bet that the loop has not yet hit its exit, which is statistically excellent when the exit is rare.

Where dynamic HLS would not help

The picture flips if `bound` is so small that the loop exits in 1 or 2 iterations: then the speculator always mispredicts, the squash cost dominates, and the design pays for the speculation hardware (Speculator + Save + Commit units, $\sim 1.5\text{--}2\times$ area per the course's experimental results) with no throughput win. Average performance is bounded below by the worst-case input, so a workload mix dominated by short-running invocations should keep the static design.

Summary

Kernel	Trick(s)	II	Latency	Total cycles
1	shift-add + pipeline + cyclic partition factor 3	1	3	$N + 2$
2	sliding-window registers + pipeline	1	6	$N + 2$
3	correctness-preserving pipeline	5	6	$5N + 1$
4	scalar acc. + rolling reg + factored mul + pipeline	1	7	$N - \text{offset} + 4$
5 (static)	nothing to do — exit cond. dominates	5	5	$5K$
5 (dynamic)	speculate on the exit condition	1	5	$K + 4$